

Laporan Eksplorasi dan Pemetaan Ruang B-202 dengan Pioneer P3DX

Penulis : Rayhan Rizqi Zamzamy
Dosen : Muhammad Qomaruz Zaman, S.T., M.T., Ph.D.
Nama Kelas : Sistem Robot Otonom
Link video YouTube : <https://youtu.be/DbYzQwSSMjk>
Link GitHub : https://github.com/hanzamzamy/SRO26_Assignment/tree/b202_explode

1. Pendahuluan

Laporan ini menyajikan desain komprehensif dari sistem navigasi otonom pada *mobile robot differential drive* Pioneer P3DX di dalam lingkungan CoppeliaSim. Tujuan utama dari tugas ini adalah melakukan pemetaan lingkungan (*mapping*) menggunakan susunan sensor ultrasonik, merancang jalur eksplorasi berbasis *grid*, dan mengimplementasikan pelacakan jalur tingkat lanjut (*advanced path tracking*). Sistem ini mengintegrasikan operasi teleoperasi manual untuk pengujian sensor dan *state-machine* otonom *sense-plan-act* secara *real-time*.

2. Pemodelan dan Algoritma Navigasi

Pemodelan dan analisis kinematika robot didasarkan pada literatur standar robotika [1], [2]. Sistem navigasi dibagi menjadi tiga subsistem utama:

- Pemetaan *grid* okupansi,
- Perencanaan jalur A* (*A-Star*), dan
- Pengendali *pure pursuit* termodifikasi.

2.1. Pemetaan Lingkungan (*Occupancy Grid Mapping*)

Pemetaan dilakukan menggunakan 9 buah sensor ultrasonik bagian depan P3DX. Ruang simulasi didiskritisasi menjadi *grid* 2D beresolusi 0.1 m/sel. Pembaruan probabilitas keberadaan rintangan menggunakan pendekatan *log-odds* untuk menghindari *underflow* numerik.

Transformasi koordinat pantulan sensor (p_S) dari kerangka lokal sensor ke kerangka dunia absolut (p_W) didefinisikan sebagai,

$$p_W = {}^W T_B \cdot {}^B T_S \cdot p_S \quad (1)$$

Dimana ${}^W T_B$ adalah matriks transformasi dari koordinat dunia ke bodi robot, dan ${}^B T_S$ adalah transformasi dari bodi ke sensor.

Sinar pantul sensor direpresentasikan ke dalam sel-sel *grid* menggunakan algoritma garis Bresenham. Pembaruan probabilitas okupansi untuk sel *grid* $m_{x,y}$ pada iterasi waktu ke- t menggunakan pembaruan Bayesian Log-Odds dirumuskan sebagai,

$$l_t(m_{x,y}) = l_{t-1}(m_{x,y}) + l_{\text{inv}}(m_{x,y} | z_t, x_t) - l_0 \quad (2)$$

Di mana l_{inv} adalah representasi *inverse sensor model* dalam ruang *log-odds*, z_t adalah pengukuran sensor, dan x_t adalah pose bodi robot pada waktu t . Nilai inversi ini didefinisikan secara komprehensif sebagai,

$$l_{\text{inv}}(m_{x,y} | z_t, x_t) = \log \left(\frac{P(m_{x,y} | z_t, x_t)}{1 - P(m_{x,y} | z_t, x_t)} \right) \quad (3)$$

Dalam implementasi praktisnya, nilai l_{inv} ditetapkan sebesar $l_{occ} = 0.85$ jika sel merupakan titik akhir pantulan (rintangan), dan $l_{free} = -0.5$ untuk setiap sel yang hanya dilalui oleh sinar (ruang bebas). Probabilitas okupansi awal (*prior*) diasumsikan seimbang, sehingga $l_0 = 0$. Untuk menyaring *noise* (*ghost obstacles*), sebuah sel hanya dideklarasikan sebagai rintangan absolut apabila total *log-odds* melampaui 1.5 (mensyaratkan minimal 2x pantulan konsektif).

2.2. Perencanaan Jalur Jarak Terpendek (A* dengan *Gradient Costmap*)

Peta tidak diperlakukan sebagai halangan biner, tetapi diekspansi (*inflation*) membentuk *gradient costmap* secara sirkular. Radius fisik robot ($r_{lethal} = 0.3$ m) diberi nilai penalti 100.0, merepresentasikan tidak dapat dilewati. Area penyangga keamanan ($r_{risk} = 0.5$ m) diberikan penalti yang menurun secara linear.

$$C_{risk} = 40.0 \times \left(1.0 - \frac{d - r_{lethal}}{r_{risk} - r_{lethal}} \right) \quad (4)$$

Algoritma A* mencari jalur optimal dengan meminimalkan fungsi biaya total $f(n)$:

$$f(n) = g(n) + h(n) + (C_{risk} \times 2.0) \quad (5)$$

Hal ini memaksa algoritma A* untuk merencanakan jalur di tengah ruangan terbuka, namun tetap mengizinkan robot masuk di lorong sempit dengan mengorbankan penalti C_{risk} jika tidak ada alternatif lain.

2.3. Pelacakan Jalur (*Pure Pursuit* dengan *Point-Turn*)

Algoritma *pure pursuit* standar tidak cocok untuk robot *differential drive* saat menghadapi tikungan patah karena algoritma tersebut didesain untuk kemudi Ackermann. Oleh karena itu, pengontrol dimodifikasi dengan penambahan batas toleransi putar-di-tempat (*point-turn*).

Koordinat target *look-ahead* (L_d) ditransformasikan ke kerangka lokal bodi robot (x_r, y_r). Kelengkungan lintasan (γ) dihitung menggunakan persamaan geometris,

$$\gamma = \frac{2y_r}{L_d^2} \quad (6)$$

Kecepatan sudut rotasi robot (ω) dikalkulasi secara proporsional terhadap kelengkungan γ . Jika target berada pada sudut tajam di atas 0.6 radian, robot akan mengubah kecepatan translasi V menjadi 0 dan berputar di tempat.

$$\omega = 1.5 \times \arctan\left(\frac{y_r}{x_r}\right) \quad (7)$$

Berdasarkan kinematika *differential drive* [1], kecepatan masing-masing roda aktuator (v_r, v_l) dengan radius roda R_w dan separuh jarak antar roda R_b adalah,

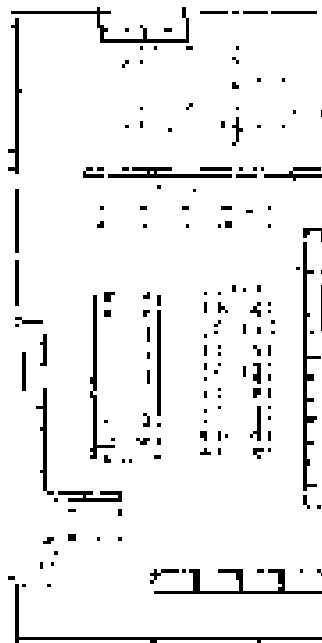
$$v_r = \frac{V + R_b\omega}{R_w}, \quad v_l = \frac{V - R_b\omega}{R_w} \quad (8)$$

3. Implementasi dan Pengujian Sistem

Subsistem pemetaan *grid* okupansi yang telah dibuat diuji menggunakan skema kendali *remote*. Robot bergerak berdasarkan *command input* dari *keyboard*. Pengujian subsistem *mapping* sangat krusial karena subsistem lainnya, seperti perancangan jalur A* dan pengendali *pure pursuit*, sangat bergantung pada akurasi peta yang dihasilkan. Poin-poin penting dalam pengujian ini di antaranya,

- Transformasi koordinat sensor yang benar,
- Pembaruan probabilitas okupansi yang konsisten, dan
- Kemampuan untuk menyaring *noise* secara efektif.

Setelah validasi pemetaan, sistem beralih ke mode otonom dengan *state-machine*.



Gambar 1: Proses pemetaan manual. Garis hitam merupakan dinding (probabilitas tinggi), area putih adalah ruang bebas.

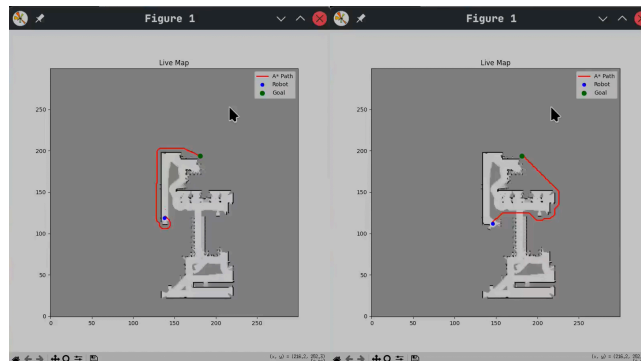
Implementasi akhir menggunakan arsitektur *state-machine* (WAITING, PLANNING, TRACKING). Antarmuka pengguna interaktif dibangun menggunakan *matplotlib* dimana pengguna dapat menentukan titik tujuan eksplorasi baru di area yang belum terpetakan.



Gambar 2: Mode otonom. Titik biru adalah posisi robot, garis merah adalah jalur A* yang sedang diikuti, titik hijau adalah tujuan eksplorasi, dan area abu-abu menunjukkan area yang belum terpetakan.

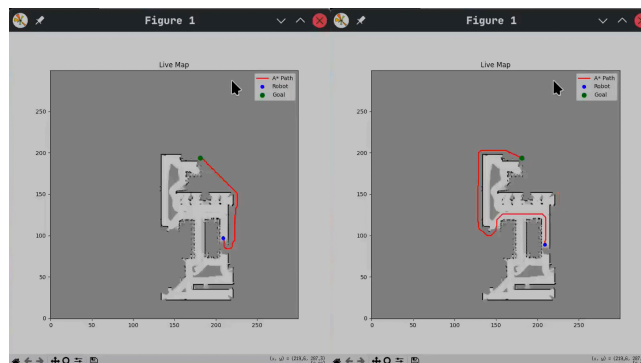
Selain itu, sistem dilengkapi dengan *software bumper*. Jika robot mendeteksi perubahan posisinya kurang dari 0.03 m dalam 1.5 detik saat motor diaktifkan, yang indikasi terjepit di dinding, robot akan mundur sejenak dan membatalkan jalur, memaksa A* melakukan *replanning*.

Peta okupansi dievaluasi oleh A* apabila ada hambatan dinamis yang muncul secara tiba-tiba di jalur yang sedang diikuti, misalnya ketika melalui area yang belum terpetakan.



Gambar 3: Kiri: Jalur A* awal (garis merah) yang direncanakan sebelum robot menemui rintangan tak terpetakan. Kanan: Jalur A* yang diperbarui secara seketika setelah robot mendeteksi rintangan, menunjukkan kemampuan *live replanning*. Pada kasus ini, jalur terpendek yang baru menghindari rintangan.

Inflasi peta okupansi yang membentuk *gradient costmap* terbukti efektif dalam memandu A* untuk merencanakan jalur yang lebih aman, meskipun terkadang mengorbankan jarak tempuh terpendek karena mengasumsikan area yang akan dilalui cukup sempit untuk dilalui.



Gambar 4: Kiri: Jalur A* awal (garis merah) yang direncanakan sebelum robot menemui rintangan celah sempit. Kanan: Jalur A* yang diperbarui secara seketika setelah robot mendeteksi rintangan. Pada kasus ini, robot memilih memutar walaupun jalur terpendek tetap bisa dilalui dengan margin yang sangat kecil.

Algoritma A* yang diimplementasikan juga terbukti mengenali *grid* yang tidak dapat dituju (misal: area di luar ruangan) setelah menjelajahi peta tertutup secara menyeluruh, sehingga tidak terjebak dalam loop perencanaan yang tidak berujung. Area di dalam ruangan masih dapat dijelajahi walaupun belum pernah dilalui sebelumnya, karena robot dapat melakukan *replanning* secara dinamis saat menemui rintangan tak terpetakan.



Gambar 5: Kiri: Robot berhenti mengejar tujuan di luar ruangan setelah memetakan seluruh area tertutup. Kanan: Robot masih dapat menjelajahi area di dalam ruangan yang belum pernah dilalui sebelumnya.

4. Kesimpulan

Sistem navigasi otonom yang dikembangkan telah memenuhi objektif desain eksplorasi robot. Implementasi *occupancy grid mapping* berbasis *log-odds* mampu menghasilkan peta biner yang tangguh terhadap *noise*. Kombinasi dari *gradient costmap* pada A* dan modifikasi *point-turn* pada *pure pursuit* terbukti sukses menyelesaikan dilema lorong sempit (*narrow passage problem*), sekaligus mengeliminasi masalah tabrakan sudut (*corner-clipping*) pada batasan sistem non-holonomik.

Daftar Pustaka

- [1] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017.
- [2] B. Siciliano and O. Khatib, *Springer Handbook of Robotics*. Springer Science & Business Media, 2008.